Team Name: Machine Minds

Course: Databases

Submission Date: April 5, 2026

Github Link: https://github.com/Praveennayak22/DBMS-PROJECT

Video Link: https://youtu.be/zgFUKo82RJ4?si=wVa5fIx7dy552oJI

2. Module A Report

ACID Transaction-Enabled B+ Tree DBMS

2.1 How correctness of operations is ensured

- Transactions have explicit begin, commit, and rollback commands. Invalid operations are rejected before finalizing.
- Uncommitted changes are kept in transaction-local storage, so they don't leak to other transactions.
- Write-Ahead Logging (WAL) records all operations before they're applied, so we can recover if there's a crash.
- B+ Tree operations (insert, delete, split, merge) are tested to ensure the tree structure stays valid.
- Integration tests run complete workflows and verify the final results are correct and match the WAL log.

2.2 How failures are handled

- If an error occurs during a transaction, rollback discards all changes and restores original values.
- Invalid operations are caught before commit, so bad data never gets written.
- Locks have timeouts, so transactions don't wait forever if there's a conflict.
- After a crash, the system replays committed transactions and rolls back incomplete ones from the WAL log.
- Tests verify that committed data survives a restart and incomplete transactions don't come back.

2.3 How multi-user conflicts are handled

- Shared locks let multiple readers access the same data at the same time.
- Exclusive locks prevent conflicting writers from accessing the same data simultaneously.
- Uncommitted changes stay invisible to other transactions (no dirty reads).
- Locks have timeouts to prevent indefinite waiting on conflicts.

- Tests verify that concurrent transactions see each other's committed changes correctly but not partial uncommitted updates.

2.4 What experiments were performed

- ACID Demos: Tests show that transactions are atomic (all-or-nothing), consistent (B+ Tree stays valid), isolated (concurrent transactions don't interfere), and durable (data survives crashes).

  What to run:
  - python demo_bplustree_atomicity.py
  - python demo_bplustree_consistency.py
  - python demo_bplustree_isolation.py
  - python demo_bplustree_durability.py

  Atomicity Results:

```
======================================================================
ATOMICITY DEMONSTRATION SUMMARY
======================================================================

✅ TEST 1: Successful COMMIT - all operations visible together
✅ TEST 2: Explicit ROLLBACK - all operations undone together
✅ TEST 3: Automatic ROLLBACK on failure - no partial results
✅ TEST 4: Mixed operations - all committed atomically

🎯 ATOMICITY PROPERTY VERIFIED FOR B+ TREE:
   ✅ All-or-nothing execution guaranteed
   ✅ No partial transaction results visible
   ✅ COMMIT makes all changes visible atomically
   ✅ ROLLBACK undoes all changes atomically
   ✅ B+ Tree maintains transactional integrity

✅ All atomicity tests PASSED
```

  Consistency Results:

```
======================================================================
CONSISTENCY DEMONSTRATION SUMMARY
======================================================================

✅ TEST 1: B+ Tree structure valid through all operations
✅ TEST 2: Constraints enforced, invalid states rejected
✅ TEST 3: ROLLBACK maintains tree validity
✅ TEST 4: Tree valid across mixed transaction outcomes

🎯 CONSISTENCY PROPERTY VERIFIED FOR B+ TREE:
  ✅ B+ Tree invariants always maintained
  ✅ Invalid states prevented by constraint checking
  ✅ Tree structure valid after COMMIT and ROLLBACK
  ✅ Consistency preserved under concurrent transactions
  ✅ B+ Tree always in valid, consistent state

✅ All consistency tests PASSED
```

Isolation Results:

```
======================================================================
ISOLATION DEMONSTRATION SUMMARY
======================================================================

✅ TEST 1: Concurrent reads isolated and consistent
✅ TEST 2: No dirty reads - uncommitted changes not visible
✅ TEST 3: Write conflicts prevented by locking
✅ TEST 4: Phantom reads prevented - consistent snapshots

🎯 ISOLATION PROPERTY VERIFIED FOR B+ TREE:
  ✅ Transactions don't interfere with each other
  ✅ No dirty reads, non-repeatable reads, or phantoms
  ✅ Proper locking prevents race conditions
  ✅ Concurrent execution is safe and correct
  ✅ Transactions appear to execute serially

✅ All isolation tests PASSED
```

Durability Results:

```
======================================================================
DURABILITY DEMONSTRATION SUMMARY
======================================================================

✅ TEST 1: Write-Ahead Logging captures all operations
✅ TEST 2: Committed data recovered after crash
✅ TEST 3: Uncommitted data NOT recovered (correct)
✅ TEST 4: Multiple transactions recovered correctly

🎯 DURABILITY PROPERTY VERIFIED FOR B+ TREE:
  ✅ Committed transactions survive crashes
  ✅ Write-Ahead Logging ensures persistence
  ✅ Recovery restores B+ Tree to consistent state
  ✅ No committed data is lost
  ✅ DURABILITY: once committed, data is permanent

✅ All durability tests PASSED
```

- **Failure Tests**: Tests inject errors into transactions, check rollback works correctly, and verify recovery from crashes.
  What to run:
  python test_comprehensive_failure_scenarios.py
  Results:

```
==========================================================================================
=======
🏁 COMPREHENSIVE FAILURE TESTING COMPLETE
==========================================================================================
=======
✅ Mid-Operation Failure Atomicity: PASSED
✅ B+ Tree Database Consistency: PASSED
✅ Crash Recovery Durability: PASSED
✅ Concurrent Failure Isolation: PASSED

📊 RESULTS: 4 PASSED, 0 FAILED
🎉 ALL FAILURE SCENARIOS HANDLED CORRECTLY!
✅ System meets all ACID requirements under failure conditions
```

- **Integration Test**: The file **test_integration_acid.py runs** all components together and confirms all ACID properties work end-to-end.

What to Run:
python test_integration_acid.py
Results:

```
======================================================================
[OK] ALL ACID PROPERTIES VERIFIED
======================================================================

Summary:
  [OK] ATOMICITY: All-or-nothing commits work correctly
  [OK] CONSISTENCY: Constraints enforced, invalid transactions rolled back
  [OK] ISOLATION: Transactions cannot see uncommitted changes
  [OK] DURABILITY: All operations logged to WAL with fsync
  [OK] CONCURRENCY: Locks prevent conflicting operations

[SUCCESS] All integration tests passed!
```

2.5 Observations and limitations

Observations

- ACID properties work correctly in both simple demos and full integration tests.
- Rollback and recovery handle failures correctly.
- Isolation prevents data corruption and dirty reads.
- B+ Tree is much faster than simple search, especially with large datasets.

Limitations

- Very long transactions may time out if other transactions are competing for locks.
- System depends on the WAL log being stored correctly on disk.
- For very small datasets, B+ Tree overhead can be slower than simple search.
- High-concurrency scenarios may need tuning of lock strategy.